EL CASO DE USO

Una taquilla con stock limitado

RESERVAR

Pido tres entradas

Si hay, se retienen a mi nombre y el stock baja.
Si no hay, me lo dice y no toca nada.

CONFIRMAR

Las pago

La reserva queda cerrada: las entradas son mías y no vuelven al stock.

CANCELAR

Cambio de idea

La reserva se anula y las entradas vuelven al stock, para el siguiente.

Haz que sea imposible vender lo que no tienes.

HOJA DE RUTA

Cinco bloques y un motor de reservas

Bloque 0 · Entorno. La máquina a punto: Go compilando y el repo clonado.

Bloque 1 · La máquina de estados. Tipos, structs, métodos y errores. `go1-step1`

Bloque 2 · El almacén. Maps, slices y la lógica de reservar, confirmar y cancelar. `go1-step2`

Bloque 3 · La taquilla, de cero. Un programa tuyo, sin esqueleto ni tests, contra tu almacén.

Cierre. Repaso de lo que has blindado, adelanto de la sesión 2 y dudas. `go1-final`

Los checkpoints se quedan: si te atascas, saltas; si no acabas hoy, acabas en casa.

BLOQUE 0 · ENTORNO

Ponemos la máquina a punto

```
git clone https://github.com/GoMalaga/workshops.git
cd workshops/go1
./check-env.sh        # valida Go 1.22 o superior
go run ./cmd/hello    # imprime tu versión de Go: estás listo
```

- Si `check-env.sh` falla: go.dev/dl, instala 1.22 o superior y vuelve a probar.
- `go test ./...` todavía no compila: al proyecto le falta un estado, a propósito. Rellenarlo es lo primero del bloque 1.

Primer checkpoint: el entorno funciona.

Un struct agrupa datos; un método, comportamiento

```go
// reservation.go
type Product struct {
    ID    string
    Name  string
    Stock int
}

type Reservation struct {
    ID        string
    ProductID string
    Quantity  int
    Status    Status
}
```

- Un `struct` es un tipo con campos con nombre: `Product` y `Reservation`, tal cual están en `reservation.go`.
- Mayúscula inicial se ve fuera del paquete; minúscula, solo dentro. No hay `public` ni `private`.
- Se crean con nombres de campo, `&Reservation{ID: "r1", Status: Held}`; lo que no nombras queda en su valor cero. Se pasan por valor; el `&` da un puntero.

Los estados son constantes, no strings

```go
// reservation.go
type Status int

const (
    Held      Status = iota // 0
    Confirmed               // 1
    Canceled                // 2
)
```

- `type Status int` crea un tipo nuevo con `int` debajo: el compilador no deja mezclar un estado con un número cualquiera.
- `iota` numera las constantes del bloque desde cero. Solo la primera lleva el tipo y el `= iota`; las demás lo heredan.
- El valor cero del tipo es `Held`: una `Reservation{}` recién creada ya está retenida.
- `String()` hace que un `Status` se imprima con su nombre, "Held", y no como el número 0.

El receptor dice si el método lee o modifica

```go
// Value receiver: s is a copy. It can only read.
func (s Status) String() string {
    switch s {
    case Held:
        return "Held"
    case Confirmed:
        return "Confirmed"
    default:
        return "Unknown"
    }
}
// Pointer receiver: r is the real one. It can change it.
func (r *Reservation) confirm() error { /* ... */ }
```

- Un método es una función con un receptor delante del nombre, `(s Status)`, y se llama `estado.String()`. No hay clases: el tipo sigue siendo solo datos.
- Receptor por valor: trabaja con una copia y sirve para leer. Receptor por puntero, `(r *Reservation)`: trabaja con el original, y es el único que puede cambiar `r.Status`.
- La llamada es igual en los dos casos, `r.confirm()`: el `&` y el `*` los pone Go.

Los errores se devuelven, no se lanzan

```go
// A sentinel: one value for the whole package.
var ErrIllegalTransition = errors.New(
    "illegal state transition",
)

func (r *Reservation) confirm() error { // the last result
    return ErrIllegalTransition // or nil when it worked
}

if err := r.confirm(); err != nil {
    return err // the caller decides
}
if errors.Is(err, ErrIllegalTransition) { ... }
```

- `error` es un valor más: la función lo devuelve como último resultado y quien llama lo mira con `if err != nil`. Sin excepciones ni `try`.
- Un error sentinela es una variable del paquete creada con `errors.New`. Se compara con `errors.Is`, nunca con el texto.
- Las reglas del dominio son errores con nombre: `ErrIllegalTransition`, `ErrOutOfStock`, `ErrProductNotFound`.

BLOQUE 1 · EJERCICIO

Haz pasar los tests de confirm() y cancel()

```
go test -run 'TestStatus|TestConfirm|TestCancel' -v
```

Has terminado el bloque cuando:

- ☐ Compila: `Canceled` está en el `iota` y en `String()`.
- ☐ `go test -run TestStatus -v` pasa.
- ☐ `go test -run 'TestConfirm|TestCancel' -v` pasa.
- ☐ Has escrito el sexto test, cancelar una reserva ya cancelada es ilegal, y pasa.

Los pasos, con detalle, en el README de go1: github.com/GoMalaga/workshops

Un map busca por clave, y la búsqueda puede fallar

```go
products := make(map[string]*Product)
products["p1"] = &Product{ID: "p1", Stock: 10}

p := products["p1"]     // *Product
ghost := products["p9"] // nil: the zero value

if _, ok := products["p9"]; !ok {
    // the key is not there
}
```

- `map[clave]valor` es la tabla hash de serie. Se crea con `make`; escribir en un map `nil` hace panic, por eso existe `NewStore()`.
- Leer una clave ausente no falla: devuelve el valor cero del tipo, `nil` para un puntero. El fallo llega al usarlo. De ahí "buscar y comprobar nil".
- El segundo valor, `ok`, dice si la clave estaba. El `Store` guarda punteros para que `p.Stock -= qty` cambie el producto real, no una copia.

El Store orquesta; la Reservation se valida sola

```go
// store.go. Given: find, guard nil, delegate.
func (s *Store) Confirm(reservationID string) error {
    r := s.reservations[reservationID]
    if r == nil {
        return ErrReservationNotFound
    }
    return r.confirm()
}
```

- `Confirm()` viene resuelto en `store.go`: buscar en el map, comprobar `nil`, delegar en el método de la reserva. `Reserve()` y `Cancel()` siguen la misma forma.
- El `Store` orquesta y toca el stock; la `Reservation` conoce sus reglas de transición. Ninguno sabe lo del otro.
- `if err := r.cancel(); err != nil { return err }` es la forma idiomática de llamar y comprobar en una línea: el error de abajo sube tal cual.

Un slice crece; un map no tiene orden

```go
names := []string{"ana", "bo"} // a slice: length 2, grows
names = append(names, "cy")    // append returns the new slice
fmt.Println(len(names), names[0]) // 3 ana

for i, n := range names { // index and value
    fmt.Println(i, n)
}
for k, v := range m { // on a map: key and value, in NO order
    fmt.Println(k, v)
}
```

- Un slice es la lista de Go: `[]T`, con `len`, que crece con `append`. `append` devuelve el slice nuevo y hay que guardarlo: `rs = append(rs, r)`.
- `range` recorre slices y maps: índice y valor, o clave y valor. `_` descarta lo que no uses.
- Un map no tiene orden: `range` lo recorre distinto en cada ejecución, a propósito. Si el orden importa, `sort.Slice` con una función de comparación, tu primer closure.

BLOQUE 2 · EJERCICIO

Haz pasar los tests de Reserve(), Cancel() y Reservations()

```
go test -run 'TestReserv|TestStore' -v
```

Has terminado el bloque cuando:

- [ ] `go test -run TestReserve_ -v` pasa: los cinco de `Reserve`.
- [ ] `go test -run TestStore -v` pasa: `Cancel` devuelve el stock, y solo una vez.
- [ ] `go test -run TestReservations -count=5 -v` pasa las cinco veces.

Los pasos, con detalle, en el README de go1: github.com/GoMalaga/workshops

BLOQUE 3 · EJERCICIO

Ahora el programa lo escribes tú

- Una taquilla por consola, en `cmd/taquilla`, sin esqueleto ni tests: lee una orden por línea y la ejecuta contra tu almacén. Reservar, confirmar, cancelar, consultar el stock y salir.
- El contrato está en el README: qué responde cada orden y una sesión de ejemplo. Si tu programa responde exactamente eso, funciona.
- Consultar el stock te obliga a añadir a `store.go` algo que el paquete todavía no ofrece desde fuera.
- Cuando funcione, prueba `reserve p1 -3` y mira el stock. Decide tu regla, escribe tu test y haz que pase.

El contrato, la sesión de ejemplo y las pistas, en el README de go1.

CIERRE

Toda la suite pasa

```
go test ./...   # los dieciocho tests pasan: has terminado
```

1

El stock nunca baja de cero

Reserve comprueba antes de restar. Reservar de más devuelve error y deja el stock intacto.

2

Solo se transiciona desde Held

Confirmed y Canceled son terminales. Cualquier otro intento es ErrIllegalTransition.

3

Lo que no existe devuelve error

Producto o reserva inexistentes: un error con nombre, nunca un panic.

Dieciocho tests que pasan: siete reglas que ya no se pueden romper.

CIERRE · LO QUE FALTA A PROPÓSITO

Una reserva olvidada bloquea stock para siempre

- Una reserva en `Held` que nadie confirma ni cancela retiene su cantidad indefinidamente.
- Treinta personas reservando y olvidándose, y la taquilla se queda sin entradas que nadie ha comprado.
- No es un olvido: es el siguiente nivel. ¿Cómo lo resuelven las webs de venta de entradas?

Hoy las reservas no caducan. La caducidad es el siguiente paso.

CIERRE · SESIÓN 2

Hoy la invariante aguanta porque nadie la ataca

```
go test -race ./...   # hoy pasa. En la sesión 2, no.
```

- Hoy nadie reserva a la vez: una llamada, un hilo, un map. La invariante no tiene con quién pelearse.
- El 12 de noviembre ponemos el `Store` detrás de una API HTTP y llegan dos peticiones a la vez a por la última entrada.
- La misma lógica, comprobar y luego restar, vende dos veces la misma entrada. Eso es el oversell, y es el tema de la sesión 2.

La sesión 2 empieza en go1-final: acaba el tuyo en casa o parte de ahí.